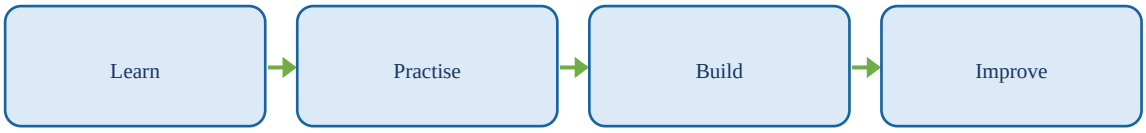
PYTHON PROGRAMMING

From Beginner to Intermediate

Your journey to becoming a programmer



A practical ebook containing explanations, illustrations, Python code, exercises, and real projects.

Designed for self-learning. Once generated, this PDF can be read completely offline.

Contents

1. Introduction to Python

2. Variables and Data Types

3. Input, Output and Operators

4. Conditions

5. Loops

6. Strings

7. Lists, Tuples, Sets and Dictionaries

8. Functions

9. Files and JSON

10. Error Handling

11. Modules

12. Object-Oriented Programming

13. Sorting and Searching

14. Money Manager Project

15. Practice Challenges

16. Quick Reference and Answers

How to learn: Read the explanation, type the code yourself, run it, change a value, and observe the result.

1. Introduction to Python

Python is a programming language used to give instructions to computers. It is useful for automation, websites, games, data analysis, and artificial intelligence.

How a program works

Your first program

```
print("Hello, world!")
print("I am learning Python.")
print(5 + 3)
```

Explanation: print() displays information. Text is placed inside quotation marks. Python can also calculate expressions such as 5 + 3.

Comments

```
# Python ignores this comment.
print("This line runs.")
```

A comment begins with #. Comments help explain your code to people who read it.

Practice exercise: Print your name, age, and favourite game.

Practice exercise: Print the result of 25 multiplied by 4.

2. Variables and Data Types

A variable stores a value under a name. You can use that value later in your program.

```
name = "Alex"
age = 12
height = 1.55
is_learning = True

print(name)
print(age)
print(height)
print(is_learning)
```

Common data types

Type	Example	Meaning
str	"Python"	Text
int	25	Whole number
float	3.14	Decimal number
bool	True	True or False

The equals sign assigns a value to a variable. Variable names are case-sensitive: score and Score are different names.

Getting input

```
name = input("Enter your name: ")
print("Welcome", name)

age = int(input("Enter your age: "))
print("Next year:", age + 1)
```

input() receives text from the user. int() converts text containing a whole number into an integer.

Practice exercise: Ask the user for two numbers and print their sum.

3. Input, Output and Operators

Operators perform calculations and compare values.

```
print(10 + 3)
print(10 - 3)
print(10 * 3)
print(10 / 3)
print(10 // 3)
print(10 % 3)
print(2 ** 3)
```

+ adds, - subtracts, * multiplies, / divides, // performs floor division, % returns the remainder, and ** calculates a power.

Comparisons

```
print(5 > 2)
print(5 == 5)
print(5 != 3)
print(4 <= 2)
```

Comparison results are True or False. Remember: = assigns a value, while == checks whether two values are equal.

Type conversion

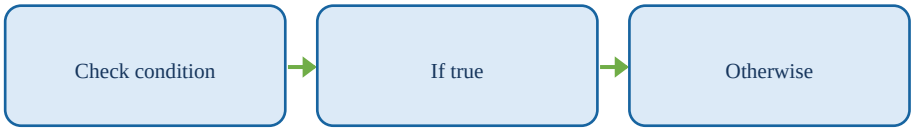
```
price_text = "25"
price = int(price_text)

print(price + 5)

temperature = float("27.5")
print(temperature)

print(str(12) + " points")
```

Practice exercise: Ask for the price of an item and its quantity, then calculate the total cost.

4. Conditions

Conditions let a program make decisions.

A decision in your program




```
marks = int(input("Enter your marks: "))

if marks >= 80:
    print("Excellent")
elif marks >= 50:
    print("Passed")
else:
    print("Keep practising")
```

if checks the first condition. elif checks another condition when earlier conditions are false. else handles the remaining cases. Indentation matters.

Combining conditions

```
age = 12
has_ticket = True

if age >= 12 and has_ticket:
    print("Entry allowed")

if age < 12 or not has_ticket:
    print("Check entry rules")
```

and requires both conditions to be true. or requires at least one to be true. not reverses True and False.

Practice exercise: Check whether a number is positive, negative, or zero.

5. Loops

Loops repeat instructions so that you do not have to write the same code many times.

For loop

```
for number in range(1, 6):
    print(number)
```

range(1, 6) produces 1, 2, 3, 4, 5. The ending number is not included.

While loop

```
count = 3

while count > 0:
    print(count)
    count -= 1

print("Finished!")
```

A while loop repeats while its condition is true. Make sure something eventually makes the condition false.

Loop through a list

```
games = ["Chess", "Roblox", "Minecraft"]

for game in games:
    print("I like", game)
```

Practice exercise: Print the multiplication table of a number.

Practice exercise: Calculate the sum of the numbers from 1 to 100.

6. Strings

A string is text. Python provides tools for searching, cleaning, and transforming text.

```
message = "  hello python  "

print(message.strip())
print(message.upper())
print(message.lower())
print(message.replace("python", "world"))
```

strip() removes spaces from the start and end. upper() makes letters uppercase. lower() makes them lowercase. replace() replaces matching text.

Indexing and slicing

```
word = "PYTHON"

print(word[0])
print(word[-1])
print(word[0:3])
```

String positions begin at zero. word[0:3] includes positions 0, 1, and 2.

Split and join

```
sentence = "red,green,blue"
colours = sentence.split(",")

print(colours)
print(" | ".join(colours))
```

Practice exercise: Ask for a full name, remove extra spaces, and print it in uppercase.

7. Lists, Tuples, Sets and Dictionaries

Collections store groups of values.

Lists

```
games = ["Chess", "Roblox", "Minecraft"]

games.append("Portal")
games.remove("Chess")

print(games)
print(games[0])
```

Lists are ordered and changeable. append() adds an item to the end. remove() removes a matching item.

Tuples and sets

```
coordinates = (10, 20)

unique_numbers = {1, 1, 2, 3, 3}

print(coordinates)
print(unique_numbers)
```

Tuples are ordered and cannot normally be changed. Sets contain unique values and do not guarantee a particular display order.

Dictionaries

```
player = {
    "name": "Alex",
    "level": 4,
    "coins": 100
}

player["coins"] += 20
player["health"] = 100

print(player["name"])
print(player["coins"])
```

A dictionary stores key-value pairs. A key such as 'coins' lets you find its associated value.

Loop through a dictionary

```
for key, value in player.items():
    print(key, "=", value)
```

Practice exercise: Create a shopping list, add two items, remove one, and print the remaining items.

Practice exercise: Create a dictionary containing a book's title, author, and publication year.

8. Functions

Functions give names to reusable instructions.

```
def greet(name):
    print("Hello", name)

greet("Sam")
greet("Asha")
```

Returning a result

```
def add(a, b):
    return a + b

answer = add(7, 5)
print(answer)
```

return sends a result back to the calling code. print() displays a value but does not replace return.

Default values

```
def discount(price, percent=10):
    return price - price * percent / 100

print(discount(200))
print(discount(200, 25))
```

Practice exercise: Write is_even(number), returning True when a number is even.

Practice exercise: Write a function that returns the largest number in a list.

9. Files and JSON

Variables disappear when a program ends. Files allow your program to save information.

Text files

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Learn Python every day.\n")

with open("notes.txt", "r", encoding="utf-8") as file:
    contents = file.read()

print(contents)
```

Mode w writes and replaces existing content. Mode a appends to the end. Mode r reads. The with statement closes the file automatically.

JSON storage

```
import json

data = {
    "name": "Alex",
    "scores": [80, 90, 95]
}

with open("data.json", "w", encoding="utf-8") as file:
    json.dump(data, file, indent=4)

with open("data.json", "r", encoding="utf-8") as file:
    loaded_data = json.load(file)

print(loaded_data["scores"])
```

JSON stores structured data such as dictionaries and lists. The json module is included with Python.

Practice exercise: Save a list of favourite games to JSON, then load it again.

10. Error Handling

Errors help you find problems in your program.

Common errors

SyntaxError: Python cannot understand the syntax.
NameError: a variable or function name is undefined.
TypeError: an operation received an unsuitable type.
ValueError: a value cannot be converted or accepted.

Handle an expected error

```
try:
    age = int(input("Enter your age: "))
except ValueError:
    print("Please enter a whole number.")
else:
    print("Your age is", age)
```

try contains code that may fail. except handles the specific error. else runs if no exception occurs.

Debugging checklist

• Read the final line of the error message.

• Check the line number in the traceback.

• Check spelling, brackets, quotation marks, and indentation.

• Print variable values to inspect what the program is doing.

• Change one thing at a time and test again.

Practice exercise: Ask for a number and handle the user entering text instead.

11. Modules

Modules contain reusable code that you can import.

```
import random
import math
from datetime import date

print(random.randint(1, 6))
print(math.sqrt(81))
print(date.today())
```

random.randint() generates a random integer. math.sqrt() calculates a square root. These modules are included with Python and work offline.

Create your own module

```
# helpers.py

def double(number):
    return number * 2

# main.py

import helpers

print(helpers.double(9))
```

Save both files in the same folder. Avoid naming your own files after standard modules such as random.py.

Practice exercise: Create a number guessing game using random.randint().

12. Object-Oriented Programming

Classes group related data and actions together.

```
class Player:
    def __init__(self, name, health=100):
        self.name = name
        self.health = health

    def take_damage(self, amount):
        self.health = max(0, self.health - amount)


hero = Player("Alex")
hero.take_damage(25)

print(hero.name)
print(hero.health)
```

__init__ runs when an object is created. self refers to the current object. Each Player object can have its own name and health.

When should you use a class?

Classes are useful when related data and actions belong together, such as a bank account with a balance and deposit method. A simple calculation may only need a function.

Practice exercise: Create a Book class with title and author attributes. Create two Book objects.

13. Sorting and Searching

Sorting orders values. Searching finds values in data.

```
scores = [45, 92, 67, 81, 54]

print(sorted(scores))
print(sorted(scores, reverse=True))

scores.sort()
print(scores)
```

sorted() returns a new sorted list. list.sort() changes the original list and returns None. Do not assign the result of list.sort() back to the list.

Sort records

```
players = [
    {"name": "Mina", "score": 82},
    {"name": "Dev", "score": 95},
    {"name": "Noor", "score": 76},
]

players.sort(
    key=lambda player: player["score"],
    reverse=True
)

for player in players:
    print(player["name"], player["score"])
```

The key argument tells Python which value to compare. Here, the score is used to sort the players.

Search a dictionary

```
cities = {
    "Ludhiana": "Punjab",
    "Jaipur": "Rajasthan",
    "Kochi": "Kerala"
}

city = input("Enter a city: ").strip().title()

if city in cities:
    print("State:", cities[city])
else:
    print("City not found.")
```

strip() removes extra spaces from the beginning and end. title() capitalises the first letter of each word. This example contains only three cities.

Practice exercise: Sort five expenses from highest to lowest.

14. Project: Personal Money Manager

Build a command-line money tracker using functions, lists, dictionaries, input validation, and JSON storage.

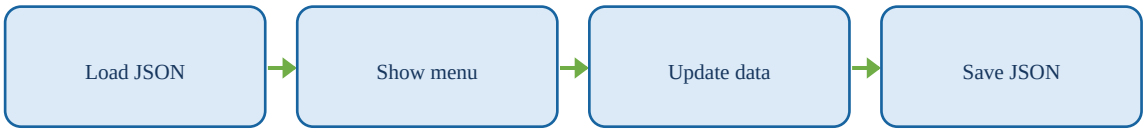
Create a file named money_manager.py. The program saves transactions in money.json in the current folder. Existing records are loaded when the program starts.

```
import json
from pathlib import Path

DATA_FILE = Path("money.json")


def load_transactions():
    if not DATA_FILE.exists():
        return []

    try:
        with DATA_FILE.open("r", encoding="utf-8") as file:
            data = json.load(file)

        if isinstance(data, list):
            return data

        return []

    except (json.JSONDecodeError, OSError):
        print("Could not read saved data.")
        return []


def save_transactions(transactions):
    with DATA_FILE.open("w", encoding="utf-8") as file:
        json.dump(transactions, file, indent=4)


def add_transaction(transactions):
    kind = input("Income or expense? ").strip().lower()

    if kind not in ("income", "expense"):
        print("Enter income or expense.")
        return

    description = input("Description: ").strip()

    try:
        amount = float(input("Amount: "))

        if amount <= 0:
            print("Amount must be greater than zero.")
            return

    except ValueError:
        print("Enter a valid number.")
        return

    transaction = {
        "type": kind,
        "description": description,
        "amount": amount
    }

    transactions.append(transaction)
    save_transactions(transactions)
```

```
    print("Transaction saved.")


def show_summary(transactions):
    income = sum(
        item["amount"]
        for item in transactions
        if item["type"] == "income"
    )

    expenses = sum(
        item["amount"]
        for item in transactions
        if item["type"] == "expense"
    )

    print(f"Income: {income:.2f}")
    print(f"Expenses: {expenses:.2f}")
    print(f"Balance: {income - expenses:.2f}")


def show_transactions(transactions):
    if not transactions:
        print("No transactions yet.")
        return

    for index, item in enumerate(transactions, start=1):
        print(
            index,
            item["type"],
            item["description"],
            f'{item["amount"]:.2f}'
        )


def main():
    transactions = load_transactions()

    while True:
        print("\n--- MONEY MANAGER ---")
        print("1. Add transaction")
        print("2. Show summary")
        print("3. Show transactions")
        print("4. Exit")

        choice = input("Choose 1-4: ").strip()

        if choice == "1":
            add_transaction(transactions)

        elif choice == "2":
            show_summary(transactions)

        elif choice == "3":
            show_transactions(transactions)

        elif choice == "4":
            print("Goodbye!")
            break

        else:
            print("Choose a number from 1 to 4.")


if __name__ == "__main__":
    main()
```

How the project works

Money manager workflow



The program loads existing records once at startup. When you add a transaction, it appends a dictionary to the list and saves the updated list. It does not reset the loaded list to an empty list.

Safety note: This is a learning project. Its JSON file is not encrypted and it is not a secure banking application.

15. Practice Challenges

Try solving these problems without copying a complete solution.

1. Calculator: add, subtract, multiply, and divide two numbers.

2. Quiz: ask five questions and calculate the final score.

3. To-do list: add, display, and remove tasks.

4. Contact book: add, search, edit, and delete contacts.

5. Expense report: calculate totals for different categories.

6. File organiser: group files by extension in a test folder.

7. Library manager: create Book objects and track borrowed books.

8. Password checker: check length and character requirements.

9. Number guessing game: provide hints and count attempts.

10. Student records: store students and sort by their marks.

16. Quick Reference and Answers

Use these examples to review important Python concepts.

Even number function

```
def is_even(number):
    return number % 2 == 0

print(is_even(8))
print(is_even(7))
```

Sum from 1 to 100

```
total = 0

for number in range(1, 101):
    total += number

print(total)  # 5050
```

Positive, negative, or zero

```
number = float(input("Enter a number: "))

if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Quick reference table

Task	Python
Display information	print(value)
Get input	input('Prompt: ')
Convert to integer	int(text)
Check a condition	if / elif / else
Repeat over items	for item in items:
Create a function	def name(parameters):
Return a result	return value
Add to a list	items.append(value)
Load JSON	json.load(file)
Save JSON	json.dump(data, file, indent=4)
Handle invalid input	except ValueError:

Final advice: You do not need to memorise everything. Learn how to break a problem into steps, test small pieces, read error messages, and build projects.

END OF EBOOK — KEEP LEARNING AND BUILDING!